

vector db notes :

Vector Embedding Search & Storage Problems

In a small project, storing and searching embeddings feels surprisingly simple. The real challenge appears when the number of vectors becomes very large.

1. Our Movie Recommendation Project Worked Because the Dataset Was Tiny

Suppose our application contains only **30 movies**.

Conceptually, our Java application could maintain:

```
List<Movie> movies;
```

Each movie contains something like:

```
class Movie {  
    String title;  
    String description;  
    float[] embedding;  
}
```

For example:

```
Interstellar  
[0.21, -0.47, 0.83, ...]  
  
Inception  
[0.18, -0.41, 0.79, ...]
```

```
Titanic  
[-0.62, 0.11, 0.22, ...]
```

Now suppose the user searches:

```
"movie about space exploration and survival"
```

We first convert the query into an embedding using the same embedding model:

```
Query  
↓  
Embedding Model  
↓  
[0.19, -0.44, 0.81, ...]
```

Then we compare the query vector with every movie vector:

```
Query ↔ Interstellar  
Query ↔ Inception  
Query ↔ Titanic  
Query ↔ Avengers  
...
```

Using something such as **cosine similarity**, we might get:

```
Interstellar → 0.94  
The Martian → 0.91  
Gravity → 0.87  
Inception → 0.72  
Titanic → 0.31
```

Sort the results by similarity and return the top few movies.

For 30 movies, this approach is completely fine.

2. What Happens When the Dataset Becomes Huge?

Now change the scale.

Spotify may have embeddings for:

```
millions of songs
```

An e-commerce platform may have embeddings for:

```
hundreds of millions of products
```

A search engine could potentially work with:

```
billions of webpages
```

Suddenly this approach:

```
List<float[]> vectors;
```

doesn't look so attractive.

We now have **two separate engineering problems**:

```
Problem 1:  
How do we STORE millions or billions of vectors?  
  
Problem 2:  
How do we SEARCH those vectors efficiently?
```

These problems are related, but they are not the same problem.

3. Why Not Simply Store Vectors in MySQL?

A common question is:

┆ We already have databases. Why do we need something different?

A normal relational table could technically look like:

```
movies
-----
id | title      | description      | embedding
-----
1  | Interstellar | Space exploration | [...]
2  | Inception   | Dream manipulation | [...]
3  | Titanic     | Romantic tragedy  | [...]
```

There is nothing that fundamentally prevents a normal database from storing an array of numbers.

A vector could be stored as:

- JSON
- binary data
- serialized data
- a dedicated vector type where supported

So the statement:

┆ Traditional databases cannot store vectors.

is incorrect.

They **can store vectors**.

The bigger challenge is:

┆ How do we efficiently search millions of vectors by similarity?

4. Traditional Database Search vs Vector Search

Traditional databases were originally designed around a different kind of question.

Consider:

```
SELECT * FROM movies
WHERE id = 10;
```

We are asking:

| Which row has an ID exactly equal to 10?

Or:

```
WHERE genre = 'Sci-Fi';
```

We are asking:

| Which rows satisfy this exact condition?

Or:

```
WHERE year >= 2010;
```

Now we are using an ordered comparison.

Traditional queries commonly involve operations such as:

```
=
>
<
BETWEEN
LIKE
JOIN
GROUP BY
```

The basic question is usually:

| Which rows satisfy my condition?

5. Vector Search Asks a Different Question

Suppose our query embedding is:

```
Q = [0.2, 0.8, 0.3]
```

We usually don't want:

```
Find a vector exactly equal to:
```

```
[0.2, 0.8, 0.3]
```

Exact equality is rarely useful for semantic search.

Instead, we want:

| Which stored vectors are closest to Q?

Consider:

```
A = [0.21, 0.79, 0.31]
```

```
B = [0.19, 0.82, 0.30]
```

```
C = [-0.7, 0.1, 0.5]
```

A and B are much closer to Q than C.

So our actual query becomes:

```
Give me the 10 vectors  
closest to Q.
```

This is a **Nearest Neighbor Search**.

6. Exact Lookup vs Similarity Lookup

Traditional Search

Query:
id = 42

Question:
Is this value equal to 42?

Result:
Yes / No

Vector Search

Query:
[0.21, -0.42, 0.75, ...]

Question:
How close is this stored vector
to the query vector?

Result:
A degree of similarity

Traditional search often looks like:

match
or
no match

Vector search looks more like:

more similar
↓
less similar

This is the key conceptual difference.

Netflix Example

Suppose a user likes **Interstellar**.

We usually don't want to search for:

```
Interstellar
```

and receive Interstellar itself.

We want:

| Movies similar to Interstellar.

That is a similarity problem rather than an exact-match problem.

7. Building the Simplest Vector Database Ourselves

Forget products such as Pinecone, Qdrant, Weaviate, Milvus, or pgvector for a moment.

Suppose we had to build the simplest possible vector-search system ourselves.

What would we need?

8. Step 1 — Store the Vectors

At minimum, we need some form of vector store.

Conceptually:

```
VectorStore
```

ID	Vector
M1	[0.21, 0.72, 0.13, ...]
M2	[0.81, 0.11, 0.45, ...]
M3	[0.18, 0.69, 0.15, ...]
M4	[...]

In practice, we would usually store more than just the vector.

For example:

```
ID:
M1

Vector:
[0.21, 0.72, 0.13, ...]

Metadata:
title = Interstellar
genre = Sci-Fi
year = 2014
```

A practical vector system therefore usually needs three conceptual pieces:

```
ID
Vector
Metadata / reference to original data
```

Why do we need the ID?

Because when a vector matches, we need to know what that vector represents.

Why do we need metadata?

Because we may eventually want queries such as:

```
Find movies similar to Interstellar

BUT only where:

genre = Sci-Fi
AND
year > 2010
```

Similarity search and traditional filtering can therefore work together.

9. Step 2 — Convert the User Query Into a Vector

Suppose the user searches:

```
"mind-bending movie about space"
```

The text itself cannot directly be compared with the stored vectors.

We first pass it through the **same embedding model**:

```
Text Query
  ↓
Embedding Model
  ↓
Query Vector
```

For example:

```
Q = [0.23, 0.70, 0.16, ...]
```

An important distinction is:

Embedding Model

```
meaning
  ↓
numbers
```

Vector Database

```
numbers
  ↓
efficient retrieval
```

The vector database is not what fundamentally understands that *Interstellar* is related to space, survival, astronauts, and science fiction.

The **embedding model creates the semantic representation**.

The **vector database searches that representation**.

10. Step 3 — Compare the Query Against Every Vector

Our simplest implementation can now calculate:

```
similarity(Q, M1)
similarity(Q, M2)
similarity(Q, M3)
similarity(Q, M4)
...
```

Suppose the scores are:

```
Interstellar = 0.95
The Martian  = 0.91
Gravity      = 0.88
Titanic      = 0.29
Notebook     = 0.21
```

Sort them:

```
0.95 → Interstellar
0.91 → The Martian
0.88 → Gravity
0.29 → Titanic
0.21 → Notebook
```

If:

```
K = 3
```

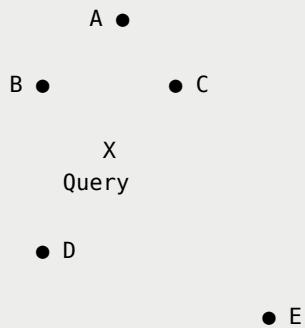
we return:

```
Interstellar  
The Martian  
Gravity
```

We have now implemented a basic **K-Nearest Neighbor Search**.

11. What Does K Mean?

Imagine the query vector as a point in space:



We want the points closest to X.

If:

```
K = 1
```

return the closest point.

If:

```
K = 3
```

return the three closest points.

If:

`K = 10`

return the ten closest points.

So:

K = how many nearest neighbors we want.

12. The Brute-Force Approach Is Correct

Suppose we have:

`100 vectors`

For every query, we perform:

`100 comparisons`

That is trivial.

But suppose we have:

`100,000,000 vectors`

Now one query could require:

`100,000,000 similarity calculations`

And that is only for **one user query**.

A large application may receive thousands of searches at the same time.

The problem quickly becomes expensive.

13. One Vector Comparison Is Also Expensive

One vector comparison is not necessarily one mathematical operation.

Suppose every embedding contains:

```
1536 dimensions
```

For cosine similarity, conceptually we process every dimension:

```
A[i] × B[i]
```

and combine those values to calculate similarity.

So the approximate brute-force work is:

```
number of stored vectors  
×  
number of dimensions
```

We can represent this roughly as:

```
O(N × D)
```

where:

```
N = number of stored vectors  
D = number of dimensions
```

14. Putting Real Numbers on the Problem

Suppose:

N = 10,000,000 vectors
D = 1536 dimensions

Conceptually:

$10,000,000 \times 1536$

That gives:

15,360,000,000

Around **15.36 billion dimension-level operations** for a naive scan, before considering practical optimizations.

And that is for a single query.

The important observation is:

| With brute-force search, the amount of work grows with the size of the dataset.

15. Storage and Memory Become Problems Too

Suppose one embedding contains:

1536 float32 numbers

One `float32` requires:

4 bytes

Therefore one vector requires approximately:

1536 × 4
= 6144 bytes
≈ 6 KB

1 Million Vectors

6 KB × 1,000,000
≈ 6 GB

Approximately 6 GB just for the raw vector values.

100 Million Vectors

≈ 600 GB

And this does not include:

IDs
metadata
indexes
database overhead
replication
temporary data

Vector search therefore creates both:

Computation problem
+
Storage / memory problem

16. Why Not Keep Everything on Disk?

One possible solution is:

Don't keep all the vectors in RAM. Store them on disk.

That helps with memory capacity, but creates another problem.

A naive query could now require:

```
Query
↓
Read huge amounts of vector data
↓
Calculate millions of similarities
↓
Find the best results
```

Accessing large amounts of data from disk or SSD for every query is much slower than working with data already available through faster memory layers.

So simply moving vectors to disk does not solve the search problem.

17. Finding the Top K Also Has a Cost

After similarity calculation, we may have:

```
M1 → 0.71
M2 → 0.23
M3 → 0.94
M4 → 0.51
...
```

Now we need the highest **K** values.

We do not necessarily need to completely sort every result. Efficient implementations can use techniques such as selection strategies or heaps.

But there is still additional work involved.

The brute-force process has three broad costs:

```
1. Read/fetch the vectors
2. Calculate similarities/distances
```

3. Find the best K results

18. Exact Nearest-Neighbor Search

Our brute-force approach has one major advantage:

We check EVERY vector.

Suppose the actual three nearest vectors are:

A
B
C

Because we check the entire dataset, we will find:

A
B
C

This is **Exact Nearest-Neighbor Search**.

Conceptually:

```
Query
↓
Search 100% of vectors
↓
Calculate all distances
↓
Return the true nearest K
```

Its trade-off is straightforward:

Excellent accuracy
↓

Poor scalability

This leads to the next important question:

If checking every vector is accurate but too expensive, can we somehow avoid checking vectors that clearly cannot be good results?

That question leads us toward **vector indexing**.

19. Why Not Use a Normal Database Index?

Databases have already solved many search-performance problems using indexes.

So why can't we simply create a normal index for embeddings?

To answer that, first understand why traditional indexes work so well.

20. Why Ordinary Indexes Work

Suppose we have:

```
Employee
-----
id
name
age
```

with 10 million rows.

We execute:

```
SELECT *
FROM employee
WHERE age = 32;
```

Without an index, the database may have to inspect many rows:

```
23
41
19
32 ← match
56
32 ← match
...
```

This is effectively a scan.

But if `age` is indexed, the database can organize the values:

```
18
20
23
27
32
36
41
50
```

Order gives us useful information.

Suppose we are searching for:

```
32
```

and currently looking at:

```
27
```

We already know:

| There is no reason to search toward the smaller values.

We can move toward the larger side.

Indexes are powerful because they allow us to **eliminate large portions of the search space without inspecting everything.**

21. Think About It in One Dimension

Imagine a number line:

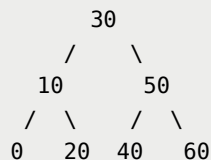
0-----10-----20-----30-----40-----50-----60

Suppose we want the value closest to:

42

There is a natural order.

A tree could organize the values like:



For 42:

```
42 > 30
→ move right

42 < 50
→ move left
```

Large parts of the tree can be discarded.

This works beautifully because numbers in one dimension have a clear order.

22. What Changes in Two Dimensions?

Now consider vectors with two dimensions:

[x, y]

For example:

A = [2, 8]
B = [5, 3]

Which vector is smaller?

Using **x**:

A.x < B.x

But using **y**:

A.y > B.y

There is no single natural ordering like the number line.

As soon as we leave one dimension, indexing becomes more complicated.

23. Partitioning Space One Dimension at a Time

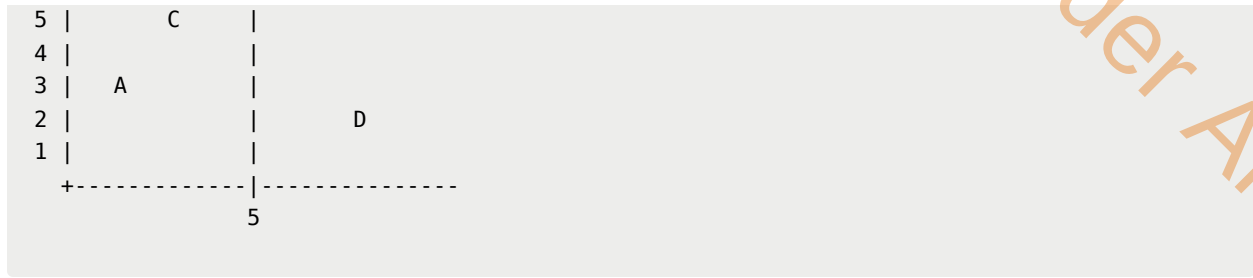
One approach is to divide the space using one coordinate at a time.

This leads to structures such as the **k-d tree**, or **k-dimensional tree**.

Suppose we first split using the **x** dimension:

x = 5

y			
9			
8	E		
7			B
6		F	



We divide the space into:

```
LEFT
x < 5

RIGHT
x >= 5
```

Conceptually:

```

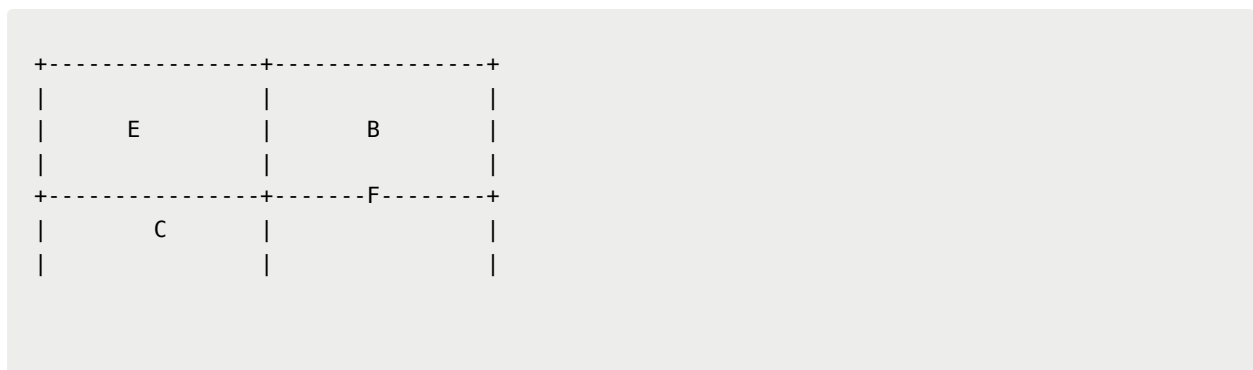
      x = 5
     /    \
    /      \
left        right
```

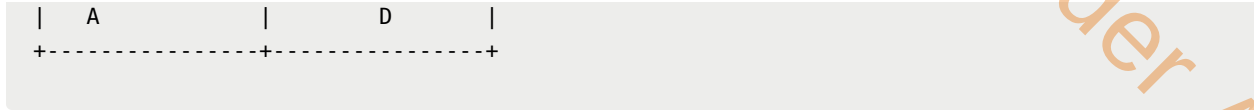
Inside each region, we can use another dimension, such as y , to split again.

```

      x = 5
     /    \
    /      \
   y = 5    y = 5
  /  \    /  \
...   ... ...   ...
```

Geometrically, the space starts getting divided into regions:





That is the basic idea behind a **k-d tree**.

24. Building a Small k-d Tree

Suppose we have:

```
A = [2,3]
B = [8,7]
C = [4,5]
D = [7,2]
E = [1,8]
F = [6,6]
```

First, sort temporarily using **x**:

```
E [1,8]
A [2,3]
C [4,5]
F [6,6]
D [7,2]
B [8,7]
```

Choose a point near the median:

```
F = [6,6]
```

Use it as the first partition:

```
      F [6,6]
      X split
     /      \
    x < 6    x > 6
```

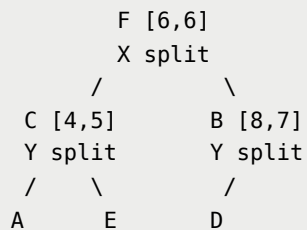
The left side contains:

E [1,8]
A [2,3]
C [4,5]

The right side contains:

D [7,2]
B [8,7]

For the next level, split using **y**:



The tree now represents geometric partitions of the vector space.

25. Searching the k-d Tree

Suppose our query is:

Q = [7,6]

Start at:

F = [6,6]

The root is currently splitting on **x**.

Compare:

```
Q.x = 7
F.x = 6

7 > 6
```

So we investigate the **right side first**.

But while navigating the tree, we also encounter:

```
F = [6,6]
```

Distance from the query:

```
Q = [7,6]
F = [6,6]

distance = 1
```

So **F** is already a good candidate.

The important question now becomes:

| Can we completely ignore the other side of the tree?

Sometimes yes.

Sometimes no.

26. When Tree Pruning Works

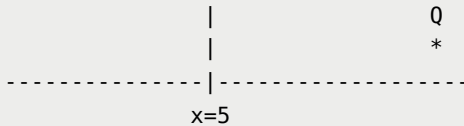
Imagine a split:

```
x < 5 | x >= 5
```

Suppose:

`Q.x = 8`

Geometrically:



The query is far from the dividing boundary.

Suppose the best vector we have already found is only:

`distance = 1`

The opposite side starts at least 3 units away in the `x` dimension alone.

Therefore:

`left side cannot contain
a closer point`

We can safely ignore that entire region.

This is called **pruning**.

Instead of searching everything:

`search RIGHT
skip LEFT`

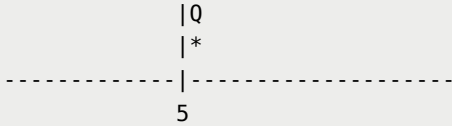
This is exactly what we want from an index.

27. When Pruning Fails

Now suppose:

`Q.x = 5.1`

The query is extremely close to the boundary:



Distance from the boundary:

`0.1`

Suppose our current nearest candidate has:

`distance = 2`

Can we ignore the left side?

No.

A point just across the boundary could easily be closer than our current result.

So we must search:

RIGHT
+
LEFT

The pruning opportunity has disappeared.

28. In Two Dimensions This Is Still Manageable

In 2D, the query may be close to an  boundary.

That might force us to search both `x` regions.

But perhaps the query is far from the relevant `y` boundaries.

We may still be able to prune other regions.

So k-d trees can still be useful in relatively low-dimensional spaces.

29. What Happens in Three Dimensions?

Now we have boundaries for:

```
x  
y  
z
```

The query has more opportunities to lie near a dividing boundary.

Whenever we cannot prove that the opposite region is irrelevant, we have to search it.

So more branches survive.

30. What Happens With 1000+ Dimensions?

Modern embeddings may contain hundreds or thousands of dimensions.

Now imagine having:

```
dimension 1 boundary  
dimension 2 boundary  
dimension 3 boundary  
...  
dimension 1000 boundary
```

It becomes increasingly difficult for the index to confidently say:

| This entire region definitely cannot contain a closer vector.

And if we cannot prove that a branch is irrelevant:

```
we have to inspect it
```

A tree that ideally behaves like:

```
ROOT
↓
B
↓
E
done
```

may start behaving more like:

```
      ROOT
     ↙   ↘
    A     B
   ↙ ↘   ↙ ↘
  C  D E  F
  ...
```

Instead of visiting only a few nodes, the search might eventually inspect:

```
70%
80%
90%
...
```

of the dataset.

At that point we are moving back toward:

```
for every vector:
    calculate distance
```

which is almost brute-force search again.

31. The Core Problem

Our journey started with a very simple algorithm:

```
Query
↓
Compare with every vector
↓
Calculate similarity
↓
Return top K
```

For a tiny dataset, this works perfectly.

At large scale, however, we face several problems:

```
Huge number of vectors
↓
Large storage requirements
↓
Millions of expensive similarity calculations
↓
Large amounts of memory/disk access
↓
Slow search
```

Traditional indexes help databases by eliminating large portions of ordered search spaces.

But vectors live in **high-dimensional spaces**, where creating useful partitions becomes much harder.

As dimensions increase, traditional geometric search structures such as k-d trees lose much of their ability to prune the search space.

Key Takeaways

- A traditional database **can store vectors**. Storage alone is not the main problem.
- Vector search asks for **similarity**, not exact equality.

- The simplest approach compares the query against every stored vector.
- Returning the nearest **K** results gives us **K-Nearest Neighbor Search**.
- Brute-force exact search scales roughly with:

$$O(N \times D)$$

where:

N = number of vectors
D = vector dimensions

- Exact search checks every vector, so it provides accurate nearest-neighbor results but scales poorly.
- Traditional indexes work well when values have a useful ordering that lets us discard large parts of the search space.
- Multi-dimensional vectors do not have the same simple ordering as scalar values.
- Structures such as **k-d trees** partition geometric space and can prune regions in lower dimensions.
- As dimensionality increases, pruning becomes harder.
- Eventually the index may inspect so much of the dataset that it begins to resemble brute-force search again.

This creates the central challenge of large-scale vector search:

How can we search only a small fraction of the vectors while still finding results that are very close to the true nearest neighbors?